

CASM: CONTEXT-AWARE SEMI-MARKOV POST-PROCESSOR FOR BEAT TRACKING

Zhanhong He^{1,2}, Hanyu Meng^{1,3}, Yaolong Ju^{1,*}

¹Great Bay University, Dongguan, China

²The University of Western Australia, Perth, Australia

³The University of New South Wales, Sydney, Australia

ABSTRACT

At the final stage of music beat tracking, neural activations need to be decoded into a discrete, musically coherent event sequence. Direct peak picking follows the local evidence but tends to produce spurious and missed detections. Dynamic Bayesian networks (DBNs), the dominant post-processing approach, suppress such errors by imposing explicit human-specified global assumptions. They work well when predefined tempo and meter assumptions match the music, but expressive or rhythmically complex pieces often depart from them, resulting in overrides correct local evidence instead of correcting it. We address this limitation by proposing CASM, a model-agnostic, context-aware semi-Markov decoder that exploits local structure in beat activations. CASM estimates local periodicity and ambiguity from the activations, then commits strongly when the cues are clear and relaxes toward the neural output when they are ambiguous, with built-in safeguards keeping this behavior stable and predictable. Using a single configuration, CASM improves F1, CMLt, and AMLt across BeatThis, MSCNN, and TCN models on GTZAN, and delivers state-of-the-art performance on SMC. Further analysis shows that CASM is less sensitive than DBNs to the choice of calibration data.¹

Index Terms— beat tracking, context-aware, semi-Markov decoder, post-processing, local periodicity

1. INTRODUCTION

Beats give music its temporal scaffold, so reliable beat tracking is crucial for music production, analysis, synchronization, and interactive systems [1]. More recently, beat tracking has become a key component of multimodal AI-generated content (AIGC), where the estimated beats serve as timing anchors for aligning generated content such as dance [2, 3], video [4, 5], and music [6, 7]. Most music beat trackers operate in two stages: a network maps audio to frame-wise beat activations, and a post-processor converts these activations into discrete events. Despite recent attempts to remove the post-processor [8–10], many contemporary systems [11–15] still rely on it to obtain musically coherent estimates. However accurate the neural activations become, direct peak picking makes isolated local decisions without considering the periodicity of beat sequences. Post-processing addresses this limitation by combining local activation evidence with the temporal regularity of music.

For post-processors, the key design question is how to balance temporal regularity against local activation evidence. A weak regularity constraint preserves the network’s predictions but may leave

missed or spurious beats uncorrected. An overly strong constraint can instead suppress valid peaks under expressive timing or rapid local tempo variation, which is common in classical music performances [16].

Many existing post-processors tilt this balance toward predefined temporal constraints. Early dynamic programming approach [17] assumes a single global tempo per track and penalizes inter-beat intervals (IBIs) that deviate from it. Conditional random fields (CRFs) [18, 19] learn sequence preferences that permit a range of local tempo variation while discouraging abrupt IBI changes. The dominant post-processor is the dynamic Bayesian network (DBN) [20, 21], which operates under pre-specified tempo ranges, meter states, and tempo transition probabilities. This design works well when these assumptions match the recording, but may require per-track or corpus-specific retuning when they do not [22].

The recently proposed PLPDP [23] shifts this balance to local evidence-conditioned temporal constraint. It derives a time-varying IBI and confidence from the predominant local pulse (PLP), which set the preferred period and how strongly it is enforced. However, this confidence is derived from PLP peak heights, it reflects how strong the selected period is but not whether a competing period is nearly as plausible. A strong period may still be nearly tied with a metrical alternative, such as half or double tempo, and enforcing it strongly then risks committing to the wrong metrical level.

We introduce CASM, a context-aware semi-Markov post-processor. The core philosophy of our approach is simple: whereas PLPDP measures periodicity strength, CASM additionally considers periodicity ambiguity. By constructing a sparse path over activation-supported peaks, CASM explicitly estimates both factors. When periodic evidence is unambiguous, CASM enforces strict temporal regularity. When competing periods create ambiguity, it weakens the temporal constraint and allows the neural activations to dominate. Bounded by a beat-count safeguard and a beat-synchronous meter decoder for deterministic performance, CASM offers four core advantages:

- **Model-agnostic.** CASM is a plug-in post-processor that operates on existing beat tracking systems and requires no neural network retraining.
- **Low tuning burden.** CASM derives its temporal constraint directly from the activations like PLPDP, requiring less per-track or per-dataset tuning than DBNs.
- **Good generalization.** A frozen deployment of CASM can handle both steady (GTZAN, common genres) and complex rhythms (SMC, deliberately challenging) across the MIREX2026 beat tracking benchmark test datasets.
- **Open and ready to use.** We release CASM as an open-source, pip-installable package with an interface compatible with the DBN post-processors in *madmom*.

*Corresponding author, juyaolong@gbu.edu.cn.

¹Code and demo are available at <https://zhanh-he.github.io/casm-beat-tracking>.

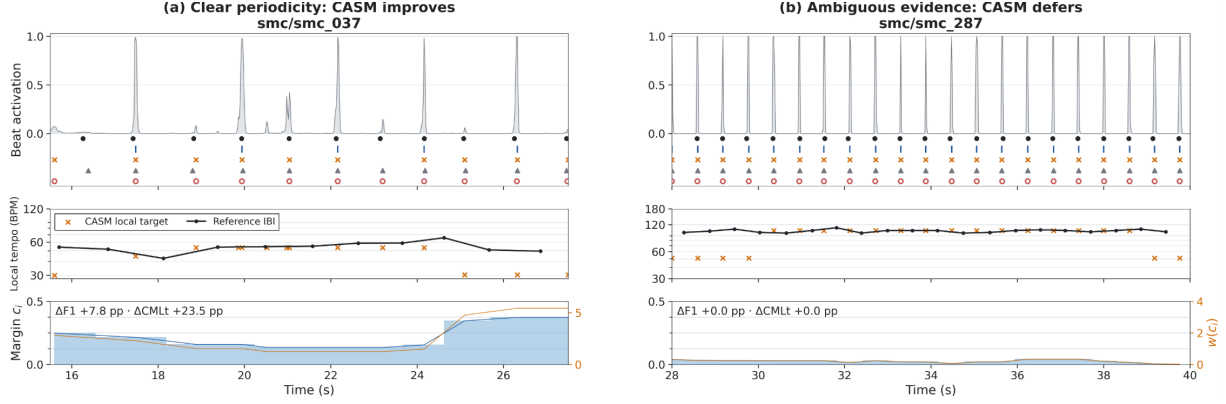


Fig. 1. CASM behaviour on held-out Beat This activations from SMC. Clear local periodicity recovers weak activation-supported beats (left), whereas ambiguous competing periods make CASM defer to Direct (right). Reference IBI is shown only for interpretation.

2. METHODOLOGY

2.1. Context-aware beat decoding

A frozen tracker produces a beat logit z_t^b at each frame t , with activation $a_t = \text{sigmoid}(z_t^b)$ and frame rate F . We retain local maxima above a fixed threshold θ_c as candidates $\mathcal{C} = \{t_1, \dots, t_N\}$, assigning each candidate a clipped evidence score e_i . For $i < j$, the elapsed time is $\Delta_{ij} = (t_j - t_i)/F$, and edge (i, j) is admissible when $60/\Delta_{ij}$ lies within 30–300 BPM. CASM selects an ordered candidate path $\pi = (i_1, \dots, i_K)$, with $i_1 < \dots < i_K$, by maximizing

$$\mathcal{S}(\pi) = \sum_{k=1}^K e_{i_k} - \sum_{k=2}^K D_{i_{k-1}, i_k} \quad (1)$$

where D_{ij} is the duration cost between candidates i and j . The two terms respectively reward strong beat evidence and penalize rhythmically inconsistent intervals. Because each transition scores a variable-duration interval, CASM has a semi-Markov structure.

At candidate i , $q_i(\ell)$ denotes the normalized autocorrelation score at an admissible lag ℓ within a window centred on t_i , including a mild fast-tempo prior. The dominant lag $\ell_i^* = \arg \max_{\ell} q_i(\ell)$ gives the local period $\tau_i = \ell_i^*/F$. We define the strongest competing hypothesis outside a ± 2 -frame neighbourhood as $q_i^{\text{alt}} = \max_{\ell: |\ell - \ell_i^*| > 2} q_i(\ell)$. The normalized period-separation margin is

$$c_i = \text{clip}_{[0,1]} \left(\frac{q_i(\ell_i^*) - q_i^{\text{alt}}}{|q_i(\ell_i^*)| + \epsilon_c} \right) \quad (2)$$

where $\epsilon_c > 0$ stabilizes the denominator. A large c_i indicates a clearly dominant period, whereas a small value indicates ambiguity, including half- or double-time alternatives. Thus, c_i is a deterministic separation margin rather than a calibrated probability. Both τ_i and c_i are median-filtered over five consecutive candidates.

For edge (i, j) , CASM combines the endpoint estimates as $\bar{\tau}_{ij} = \sqrt{\tau_i \tau_j}$ and $c_{ij} = \sqrt{c_i c_j}$. The ambiguity-conditioned constraint strength is $g(c) = \lambda c / \{2[\sigma_0 + (1 - c)\sigma_u]^2\}$, giving the duration cost

$$D_{ij} = g(c_{ij}) \log^2 \left(\frac{\Delta_{ij}}{\bar{\tau}_{ij}} \right) \quad (3)$$

Here, $\lambda > 0$ controls the overall penalty strength, $\sigma_0 > 0$ sets the minimum timing tolerance, and $\sigma_u \geq 0$ controls relaxation under

ambiguity. The cost vanishes when $\Delta_{ij} = \bar{\tau}_{ij}$; a large c_{ij} enforces rhythmic regularity, while a small c_{ij} relaxes the constraint. CASM can therefore recover weak but rhythmically supported beats without forcing an unreliable tempo grid.

Figure ?? shows how Eq. (3) converts the local period-separation margin into an input-dependent timing constraint. Panel (a) shows the fixed response, panel (b) maps transitions from different datasets and backbones onto it, and panel (c) summarizes the resulting track-level constraint strengths. Thus, one frozen CASM configuration adapts its constraint across inputs. The figure illustrates decoder conditioning, not tracking accuracy; the direct fallback is a separate safeguard.

2.2. Safeguards and downbeat decoding

A Viterbi-style dynamic program maximizes Eq. (1) and backtracks the best beat path, allowing a restart after an unfavourable prefix. As a safeguard, CASM also forms a direct fallback path using seven-frame max pooling, a zero-logit threshold, and one-frame deduplication [9]. If $|\pi_{\text{CASM}}|/|\pi_{\text{direct}}| \notin [r_{\min}, r_{\max}]$, indicating an implausibly sparse or dense CASM path, the direct path is returned. This rule is deterministic and requires no learned router.

Downbeats are decoded on the selected beat grid using a second dynamic program that favours valid bar lengths and discourages unnecessary meter changes. If its output strongly disagrees with the direct downbeat activations snapped to the same grid, the snapped result is used, ensuring that every downbeat coincides with an output beat. All scalar parameters are calibrated once on the development folds and then frozen across tracks, datasets, and backbone models. Inference requires no reference annotations, externally supplied BPM or meter, per-track tuning, or learned decoder parameters.

3. EXPERIMENTS

3.1. Evaluation protocol

Datasets and splits. We use the publicly released data from BeatThis [9], which contains 4,556 tracks from 18 beat-tracking datasets, including SMC, Hainsworth, Ballroom, Beatles, and others. This data collection provides preprocessed log-Mel spectrograms computed from 22.05kHz mono audio at 50 fps, together with a fixed eight-fold partition to conduct cross-validation experiments. For each fold, the corresponding backbone checkpoint is

trained on the remaining seven folds, ensuring that evaluated tracks remain unseen during training. Among these datasets, SMC provides beat but not downbeat annotations, and is widely recognized as the hard one compared to the others. Additionally, GTZAN provides 993 tracks as the held-out evaluation set, which is excluded from the cross-validation pool.

Backbones. We select three models whose implementations are publicly available and readily reproducible. For BeatThis, we use the original code and pretrained checkpoints [9]. MSCNN is adapted from the multi-scale, multi-task convolutional network of [24], which was designed for 60s Bark-scale specific-loudness features. We modify it to accept the 30s the unified log-Mel input and remove the task heads unrelated to beat and downbeat tracking. TCN was introduced by [25], and we use the re-implementation distributed with [24], which has been adapted from its original 5 s input to the same 30 s log-Mel input, with the tempo task head removed. All resulting backbones are trained on the same data and frozen before decoding performance comparison, ensuring that every post-processor receives exactly the same 50 Hz framewise beat/downbeat activations from a given backbone.

Evaluation. We conduct the same eight-fold cross-validation as in [9], where backbones are each evaluated on the held-out GTZAN and their corresponding out-of-fold SMC partitions. We use `mir_eval` [26] to compute F1 score with a ± 70 ms tolerance, CMLt, and AMLt. CMLt measures continuity at the correct metrical level, whereas AMLt additionally accepts standard off-beat and half- or double-level interpretations.

3.2. Post-processors and tempo support

As we defined our task is offline, the future information is accessible for model to top up the estimate accuracy. We compare CASM with the direct decoding, along with other offline post-processing methods including DBN [20], PLPDP [23], and CRF beat detection [27]. Direct is the peak-picking path defined in Sec. 2, and CASM uses its fixed 30–300-BPM support. PLPDP, CombFilter, and CRF are run with their released/default parameters. For methods without a joint beat/downbeat decoder, downbeats are decoded from the downbeat activation stream and snapped to the corresponding beat grid.

DBN receives separate treatment because its behaviour is particularly sensitive to global tempo support. We use the standard joint `madmom` configuration [20]: meters {3, 4}, transition weight 100, observation weight 16, threshold 0.05, and the default 55–215-BPM range. Since many SMC data are fall below 55 BPM [22], Table 1 also explores a 30–300-BPM DBN variant in which only the tempo range is changed. For a matched diagnostic, we additionally report CASM with its support restricted to 55–215 BPM on BeatThis activations, with all other CASM settings remain unchanged.

3.3. CASM calibration

CASM has no learned weights or random seed. The final configuration used in all main tables is calibrated once on the union of SMC folds 1–7 (7F) and then frozen for every track, backbone, and dataset. The same global scalar settings are therefore used on BeatThis, MSCNN, and TCN, on both SMC and GTZAN, without per-track or per-dataset BPM, meter, or parameter tuning at inference. Because the aggregate SMC results use eight-fold out-of-fold backbone activations while the decoder configuration is calibrated on folds 1–7, they are out-of-fold with respect to backbone training but are not a nested estimate of decoder calibration.

We analyse calibration-data sensitivity separately rather than using it to choose the main operating point. SMC fold 0 is held out from decoder calibration throughout. From folds 1–7, we enumerate every 1-, 2-, and 4-fold calibration subset (7, 21, and 35 combinations) and the seven-fold union. For both CASM and DBN, a decoder configuration is selected independently on each calibration subset and then evaluated on the same fixed SMC-fold-0 panel and on GTZAN using one fixed backbone seed. The resulting distributions therefore measure sensitivity to the amount and composition of calibration data, while the 7F CASM configuration is the maximally supported global configuration used for the main results.

4. RESULTS

4.1. Comparison with post-processors

Table 1 isolates the post-processor by holding the backbone activations fixed. With 30–300-BPM support, the CASM improves BeatThis (compared with direct pick-peaking) on every reported metrics across all three backbones and both datasets. Relative to DBN, CASM generally preserves F1 better, while DBN can produce larger continuity gains, particularly for GTZAN downbeats. The large SMC change between the 55–215- and 30–300-BPM DBN variants also shows how strongly a fixed DBN can depend on its global tempo range. PLPDP, CombFilter, and CRF provide useful alternatives, but none improves every reported metric under the same fixed activations. Overall, CASM gives a conservative F1–continuity operating point that transfers across backbone families without changing decoder parameters.

Table 1. Post-processing comparison on fixed backbone activations. Backbone-only rows use direct peak picking. Post-processors use 30–300 BPM unless marked ^[dbn], which denotes the 55–215 BPM range optimized for DBN in GTZAN-like music.

Method	GTZAN (Beat / Downbeat)						SMC (Beat)		
	F1	CMLt	AMLt	F1	CMLt	AMLt	F1	CMLt	AMLt
BeatThis (direct) [9]	89.1	79.8	89.8	78.3	67.2	79.1	62.7	51.4	61.0
w. CASM	+0.3	+0.8	+1.2	+0.7	+3.6	+8.0	+0.3	+2.4	+3.6
w. DBN [20]	−0.5	−0.9	+0.2	−1.5	−0.6	+1.1	−1.6	+0.4	+3.5
w. DBN ^[dbn]	−1.0	+0.7	+1.3	−0.9	+6.2	+8.7	−5.2	−3.9	+1.5
w. PLPDP [23]	+0.1	+0.3	+0.3	+0.1	+0.2	+0.2	−0.9	−3.5	−0.3
w. CRF [27]	−0.6	−0.3	+1.4	−1.4	+1.9	+5.5	−1.2	+1.4	+1.2
MSCNN [24]	86.1	73.4	79.9	70.8	53.1	63.1	57.2	42.0	48.4
w. CASM	+0.4	+1.6	+2.4	+1.5	+12.5	+18.3	+0.3	+3.5	+6.8
w. DBN	−1.1	+0.2	+5.9	−0.9	+9.5	+14.6	−0.2	+2.2	+4.1
w. DBN ^[dbn]	−0.6	+4.0	+7.2	+1.8	+15.8	+21.2	−4.9	+0.3	+3.9
TCN [25]	87.1	73.8	81.7	62.4	24.6	59.9	52.9	30.1	36.0
w. CASM	+0.3	+1.3	+1.6	+2.1	+9.2	+13.4	+1.7	+4.5	+8.2
w. DBN	−1.9	−0.8	+4.4	−0.2	+8.9	+9.3	−0.4	+2.1	+5.9
w. DBN ^[dbn]	−2.6	−0.7	+5.0	+4.9	+23.2	+22.9	−0.9	−7.0	+2.3

4.2. Ablation study

Table 2 shows that the main CASM components contribute to a balanced F1–continuity trade-off. Fixing the local-target precision substantially reduces the gains, particularly on GTZAN, while using only periodicity strength or width also degrades Beat F1 and weakens continuity improvements. This supports combining complementary local periodicity cues rather than relying on a single confidence measure. Using one endpoint produces results close to the full CASM, suggesting that this design choice has a relatively small effect. Removing the safeguards can yield slightly larger CMLt or

AMLt gains, but at the cost of Beat F1, most clearly on SMC (+0.2 to −0.3). Overall, the full CASM provides the most consistent balance between preserving local beat evidence and improving temporal continuity across both datasets.

Table 2. Ablation study of CASM on BeatThis activations. All values are changes relative to the direct backbone output.

Method	GTZAN (Beat)			SMC (Beat)		
	F1	CMLt	AMLt	F1	CMLt	AMLt
CASM (default, 30–300 BPM)	+0.3	+0.8	+1.2	+0.3	+2.4	+3.6
Change to 55–215 BPM	+0.1	+0.5	+0.6	+0.1	+0.4	+2.0
Local target fixed precision	−0.3	−0.8	−0.8	+0.1	+0.7	+0.9
Strength only	−0.1	−0.2	−0.2	−0.5	+0.9	+2.1
Width only	−0.3	−0.6	−0.6	−0.5	+0.9	+2.2
One endpoint	+0.1	+0.3	+0.4	+0.2	+2.2	+2.6
No safeguards	−0.1	+0.4	+0.5	−0.3	+2.5	+2.7

4.3. Comparison with the state of the art

Table 3 compares the BeatThis works with CASM results with recent literature. While using CASM improves six beat/downbeat metrics compared to BeatThis (directly peak-picking) on the GTZAN, we reported the highest scores on SMC compared to the other SOTA systems. On GTZAN, stronger task-specific or pre-trained frontends retain the best individual scores. The literature table therefore provides system-level context; controlled claims about decoder behaviour come from Table 1, where the backbone activations are held fixed.

Table 3. Comparison with recent beat-tracking systems. BeatThis (direct) and CASM results were obtained from our experiments, while all other values are reported from their original papers.

Method	GTZAN (Beat / Downbeat)						SMC (Beat)		
	F1	CMLt	AMLt	F1	CMLt	AMLt	F1	CMLt	AMLt
HN-MERT w.DBN [12]	89.7	81.4	94.3	77.4	71.6	87.2	61.7	—	—
HN-MusicFM w.DBN	89.2	80.9	93.7	79.8	73.2	89.5	60.6	—	—
BeatFM-MERT w.DBN [13]	89.5	82.7	93.9	76.7	70.2	85.4	61.3	—	—
BeatFM-MusicFM w.DBN	89.1	80.6	93.5	<u>79.6</u>	<u>74.4</u>	<u>88.7</u>	60.5	—	—
BeatMamba w.DBN [14]	88.9	82.3	94.3	—	—	—	58.7	46.8	62.4
BeatKAN w.DBN [15]	88.2	78.1	92.3	—	—	—	59.8	48.4	64.0
BeatThis-MDM [28]	89.7	82.9	92.5	79.5	76.4	88.5	61.0	48.9	<u>64.3</u>
BeatThis [9]	89.1	79.8	89.8	78.3	67.2	79.1	<u>62.7</u>	51.4	61.0
BeatThis w.CASM	89.4	80.6	91.0	79.0	70.8	87.1	63.0	53.8	64.6

4.4. Sensitivity to calibration data

The sensitivity experiment keeps the evaluation panels fixed and changes only the data used to calibrate the decoder. For each calibration scale $k \in \{1, 2, 4, 7\}$, CASM and DBN are calibrated on the corresponding SMC fold subsets defined in Sec. 3, and every selected configuration is evaluated on the same held-out SMC fold 0 and the same GTZAN backbone seed. Thus, the spread at a given scale measures dependence on which calibration folds were used, while the movement of the distribution centre measures how additional calibration data changes transfer to the two fixed evaluation panels.

Fig. 2 shows a clear difference in calibration behaviour. CASM remains tightly clustered across fold combinations, and its central performance improves consistently as more calibration folds are used on both the held-out SMC panel and GTZAN. In contrast, DBN exhibits much wider variation across calibration subsets, and

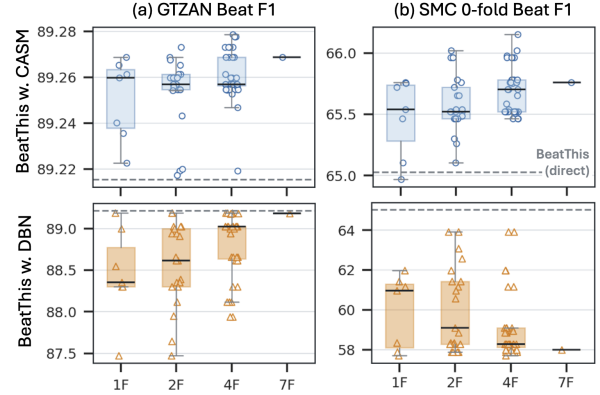


Fig. 2. Sensitivity to calibration data for CASM and DBN. Each point corresponds to a decoder configuration selected from one of the 7/21/35 exhaustive 1-/2-/4-fold subsets of SMC folds 1–7 and evaluated on fixed SMC-fold-0 and GTZAN panels; 7F is a single configuration. Boxes are descriptive because calibration subsets overlap, and dashed lines show Direct performance.

the trends on SMC and GTZAN do not move together: configurations favoured by one panel can transfer poorly to the other. This cross-dataset disagreement is consistent with stronger calibration-set over-specialisation in the fixed DBN parameterisation. The 7F CASM setting used in the main tables is therefore supported by the full available calibration pool rather than selected from a favourable smaller-fold combination.

5. CONCLUSION AND FUTURE WORK

CASM revisits structured beat decoding as an inductive bias conditioned on each input, not as a return to a fixed global grid. It joins activation-supported events with locally centred segment potentials and deliberately relaxes those potentials when periodic evidence is ambiguous. Once globally calibrated, the same deterministic module runs without upstream retraining, user annotation, or per-track tempo and meter settings. Current evidence supports a conservative F1–continuity alternative to Direct and DBN decoding, not universal dominance. The next version should use strict nested or leave-one-dataset-out calibration, matched tempo-support baselines, PLPDP and mechanism ablations, and explicit safeguard statistics. Multi-hypothesis period paths, causal decoding, and a differentiable duration layer are natural extensions, as is evaluation on every recent tracker that releases compatible activations.

Acknowledgments. The computational resources are supported by SongShan Lake HPC Center (SSL-HPC) at Great Bay University.

6. REFERENCES

- [1] Hendrik Purwins, Bo Li, Tuomas Virtanen, Jan Schlüter, Shuo-Yiin Chang, and Tara Sainath, “Deep learning for audio signal processing,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 13, no. 2, pp. 206–219, 2019.
- [2] Jonathan Tseng, Rodrigo Castellon, and C. Karen Liu, “EDGE: Editable dance generation from music,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 448–458.

- [3] Zikai Huang, Xuemiao Xu, Cheng Xu, Huaidong Zhang, Chenxi Zheng, Jing Qin, and Shengfeng He, “Beat-It: Beat-synchronized multi-condition 3D dance generation,” in *Proc. Euro. Conf. Comput. Vision (ECCV)*, 2024, pp. 273–290.
- [4] Jiashuo Yu, Yaohui Wang, Xinyuan Chen, Xiao Sun, and Yu Qiao, “Long-term rhythmic video soundtracker,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023, pp. 40339–40353.
- [5] Xinyi Tong, Yiran Zhu, Jishang Chen, Chunru Zhan, Tianle Wang, Sirui Zhang, Nian Liu, Tiezheng Ge, Duo Xu, Xin Jin, Feng Yu, and Song-Chun Zhu, “Video echoed in music: Semantic, temporal, and rhythmic alignment for video-to-music generation,” in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2026, vol. 40, pp. 25983–25991.
- [6] Yun-Han Lan, Wen-Yi Hsiao, Hao-Chung Cheng, and Yi-Hsuan Yang, “MusiConGen: Rhythm and chord control for transformer-based text-to-music generation,” in *Proc. ISMIR*, 2024.
- [7] Ke Chen, Yusong Wu, Haohe Liu, Marianna Nezhurina, Taylor Berg-Kirkpatrick, and Shlomo Dubnov, “MusicLDM: Enhancing novelty in text-to-music generation using beat-synchronous mixup strategies,” in *Proc. ICASSP*, 2024, pp. 1206–1210.
- [8] Tsung-Ping Chen and Li Su, “Toward postprocessing-free neural networks for joint beat and downbeat estimation,” in *Proc. ISMIR*, 2022, pp. 27–35.
- [9] Francesco Foscari, Jan Schlüter, and Gerhard Widmer, “Beat this! accurate beat tracking without DBN postprocessing,” in *Proc. ISMIR*, 2024, pp. 962–969.
- [10] James Bolt, Johan Pauwels, and George Fazekas, “Beat and downbeat detection: A reformulated approach,” in *Proc. ICASSP*, 2026, pp. 15282–15286.
- [11] Jiajun Deng, Yaolong Ju, Jing Yang, Simon Lui, and Xunying Liu, “Efficient adapter tuning for joint singing voice beat and downbeat tracking with self-supervised learning features,” in *Proc. ISMIR*, 2024.
- [12] Ganghui Ru, Jieying Wang, Jiahao Zhao, Yulun Wu, Yi Yu, Nannan Jiang, Wei Wang, and Wei Li, “HingeNet: A harmonic-aware fine-tuning approach for beat tracking,” in *Proc. IEEE Int. Conf. Multimedia Expo (ICME)*, 2025, pp. 1–6.
- [13] Ganghui Ru, Jieying Wang, Jiahao Zhao, Yulun Wu, Yi Yu, Nannan Jiang, Wei Wang, and Wei Li, “BeatFM: Improving beat tracking with pre-trained music foundation model,” in *Proc. IEEE Int. Conf. Multimedia Expo (ICME)*, 2025, pp. 1–6.
- [14] Ganghui Ru, Yi Yu, and Wei Li, “BeatMamba: Bidirectional selective state-space modeling for efficient beat tracking,” in *Proc. ICASSP*, 2026.
- [15] Zhicheng Zhang, Ganghui Ru, and Wei Li, “BeatKAN: An efficient and drum-attuned beat tracking method using kolmogorov-arnold networks,” in *Proc. ICASSP*, 2025, pp. 1–5.
- [16] Hendrik Schreiber, Frank Zalkow, and Meinard Müller, “Modeling and estimating local tempo: A case study on chopin’s mazurkas,” in *Proc. ISMIR*, 2020, pp. 773–779.
- [17] Daniel P. W. Ellis, “Beat tracking by dynamic programming,” *Journal of New Music Research*, vol. 36, no. 1, pp. 51–60, 2007.
- [18] Thomas Fillon, Cyril Joder, Simon Durand, and Slim Essid, “A conditional random field system for beat tracking,” in *Proc. ICASSP*, 2015, pp. 424–428.
- [19] Magdalena Fuentes, Brian McFee, Hélène C. Crayencour, Slim Essid, and Juan Pablo Bello, “A music structure informed downbeat tracking system using skip-chain conditional random fields and deep learning,” in *Proc. ICASSP*, 2019, pp. 481–485.
- [20] Florian Krebs, Sebastian Böck, and Gerhard Widmer, “An efficient state-space model for joint tempo and meter tracking,” in *Proc. ISMIR*, 2015, pp. 72–78.
- [21] Sebastian Böck, Florian Krebs, and Gerhard Widmer, “Joint beat and downbeat tracking with recurrent neural networks,” in *Proc. ISMIR*, 2016, pp. 255–261.
- [22] Jaehoon Ahn, Tae Gum Hwang, and Moon-Ryul Jung, “The SMC blind spot: A failure mode analysis of state-of-the-art beat tracking,” 2026, <https://arxiv.org/abs/2605.12287>.
- [23] Ching-Yu Chiu, Meinard Müller, Matthew E. P. Davies, Alvin Wen-Yu Su, and Yi-Hsuan Yang, “Local periodicity-based beat tracking for expressive classical piano music,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 31, pp. 2824–2835, 2023.
- [24] Zhanhong He, Hanyu Meng, Defeng Huang, and Roberto Togneri, “Joint estimation of piano dynamics and metrical structure with a multi-task multi-scale network,” in *Proc. ICASSP*, 2026, pp. 14607–14611.
- [25] Sebastian Böck and Matthew E. P. Davies, “Deconstruct, analyse, reconstruct: How to improve tempo, beat, and downbeat estimation,” in *Proc. ISMIR*, 2020, pp. 574–582.
- [26] Colin Raffel, Brian McFee, Eric J. Humphrey, Justin Salamon, Oriol Nieto, Dawen Liang, and Daniel P. W. Ellis, “mir.eval: A transparent implementation of common MIR metrics,” in *Proc. ISMIR*, 2014, pp. 367–372.
- [27] Filip Korzeniowski, Sebastian Böck, and Gerhard Widmer, “Probabilistic extraction of beat positions from a beat activation function,” in *Proc. ISMIR*, 2014, pp. 513–518.
- [28] Francesco Foscari, Filip Korzeniowski, and Richard Vogl, “Masked diffusion enables coherent beat tracking,” in *Proc. ISMIR*, 2026.